



EC200U&EG91xU Series QuecOpen **SMS API Reference Manual**

LTE Standard Module Series

Version: 1.1

Date: 2023-09-04

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-02-24	Marvin NING	Creation of the document
1.0	2021-10-18	Marvin NING	First official release
1.1	2023-09-04	Marvin NING	Added the applicable modules EG912U-GL and EG915U series.

Contents

About the Document.....	3
Contents.....	4
Table Index.....	5
1 Introduction	6
1.1. Applicable Modules.....	6
2 SMS APIs	7
2.1. Header File.....	7
2.2. API Functions Overview.....	7
2.3. API Functions Description.....	8
2.3.1. ql_sms_get_init_status.....	8
2.3.1.1. ql_sms_errcode_e.....	8
2.3.2. ql_sms_send_msg	9
2.3.2.1. ql_sms_code_e	10
2.3.3. ql_sms_send_pdu	10
2.3.4. ql_sms_read_msg.....	11
2.3.4.1. ql_sms_format_e.....	11
2.3.5. ql_sms_read_msg_list	12
2.3.6. ql_sms_delete_msg	12
2.3.7. ql_sms_get_center_address.....	13
2.3.8. ql_sms_set_center_address	13
2.3.9. ql_sms_set_storage	14
2.3.9.1. ql_sms_stor_e	15
2.3.10. ql_sms_get_storage.....	15
2.3.10.1. ql_sms_mem_info_t.....	16
2.3.11. ql_sms_get_storage_info	16
2.3.11.1. ql_sms_stor_info_s	17
2.3.12. ql_sms_callback_register.....	18
2.3.12.1. ql_sms_event_handler_t.....	18
2.3.12.2. ql_sms_event_id_e	19
3 SMS Demo	20
3.1. Demo Introduction.....	20
3.2. Demo Autorun	22
4 Appendix References	23

Table Index

Table 1: Applicable Modules	6
Table 2: API Functions Overview	7
Table 3: Related Document	23
Table 4: Terms and Abbreviations	23

1 Introduction

Quectel LTE Standard EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS system, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document mainly introduces the SMS APIs of EC200U series and EG91xU family modules and related examples in QuecOpen® solution. SMS APIs in this document can mainly realize features listed below:

- Send SMS messages
- Receive SMS messages
- Delete SMS messages
- Get SMS center number
- Get the storage status of SMS messages

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 SMS APIs

2.1. Header File

ql_api_sms.h, the header file of SMS APIs, is located in the directory of *ql-sdk\components\ql-kernel\inc* in SDK. Unless otherwise specified, the header files mentioned in this document are all located in this directory.

2.2. API Functions Overview

Table 2: API Functions Overview

Function	Description
<i>ql_sms_get_init_status()</i>	Gets the initialization status of SMS
<i>ql_sms_send_msg()</i>	Sends SMS message in text format
<i>ql_sms_send_pdu()</i>	Sends SMS message in PDU format
<i>ql_sms_read_msg()</i>	Reads SMS message in text or PDU format
<i>ql_sms_read_msg_list()</i>	Reads the list of SMS messages
<i>ql_sms_delete_msg()</i>	Deletes SMS messages
<i>ql_sms_get_center_address()</i>	Gets the SMS center number
<i>ql_sms_set_center_address()</i>	Sets the SMS center number
<i>ql_sms_set_storage()</i>	Sets the storage location of SMS message
<i>ql_sms_get_storage()</i>	Gets storage location of SMS message
<i>ql_sms_get_storage_info()</i>	Gets the storage status of SMS message in (U)SIM card and ME
<i>ql_sms_callback_register()</i>	Registers the SMS message callback function

2.3. API Functions Description

2.3.1. ql_sms_get_init_status

This function gets the initialization status of SMS. The SMS related functions can only be called after initialization.

● Prototype

```
ql_sms_errcode_e ql_sms_get_init_status(uint8_t nSim, uint8_t *status)
```

● Parameter

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

status:

[Out] The initialization status of SMS.

0 Uninitialized

1 Initialized

● Return Value

See **Chapter 2.3.1.1** for details.

2.3.1.1. ql_sms_errcode_e

The enumeration of SMS APIs execution error codes is defined as follows:

```
typedef enum
{
    QL_SMS_SUCCESS                = 0,
    QL_SMS_ERROR                  = 1 | (QL_COMPONENT_SMS << 16),
    QL_SMS_NOT_INIT_ERR,
    QL_SMS_PARA_ERR,
    QL_SMS_NO_MEMORY_ERR,
    QL_SMS_SEM_CREATE_ERR,
    QL_SMS_SEM_TIMEOUT_ERR,
    QL_SMS_NO_MSG_ERR,
}ql_sms_errcode_e
```

- **Parameter**

Parameter	Description
<i>QL_SMS_SUCCESS</i>	Successful execution
<i>QL_SMS_ERROR</i>	Failed execution
<i>QL_SMS_NOT_INIT_ERR</i>	SMS service is not initialized
<i>QL_SMS_PARA_ERR</i>	Parameter error
<i>QL_SMS_NO_MEMORY_ERR</i>	No memory
<i>QL_SMS_SEM_CREATE_ERR</i>	Fail to create semaphore
<i>QL_SMS_SEM_TIMEOUT_ERR</i>	Semaphore times out
<i>QL_SMS_NO_MSG_ERR</i>	No SMS messages to read

2.3.2. ql_sms_send_msg

This function sends SMS messages in text format.

- **Prototype**

```
ql_sms_errcode_e ql_sms_send_msg(uint8_t nSim, char *phone_num, char *data, ql_sms_code_e code)
```

- **Parameter**

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

phone_num:

[In] The recipient's mobile phone number.

data:

[In] Message content. Length of the message content is less than 140 bytes.

code:

[In] SMS encoding method. See **Chapter 2.3.2.1** for details.

- 0 GSM-7 bit
- 1 UCS2

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.2.1. ql_sms_code_e

The SMS encoding method is defined as follows:

```
typedef enum
{
    GSM = 0,
    UCS2 = 1,
}ql_sms_code_e
```

- **Parameter**

Parameter	Description
<i>GSM</i>	GSM encoding
<i>UCS2</i>	UCS2 encoding, used when the sent message contains Chinese.

2.3.3. ql_sms_send_pdu

This function sends SMS messages in PDU format.

- **Prototype**

```
ql_sms_errcode_e ql_sms_send_pdu(uint8_t nSim, char *pdu)
```

- **Parameter**

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

pdu:

[In] PDU message.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.4. ql_sms_read_msg

This function reads SMS messages in text or PDU format.

● Prototype

```
ql_sms_errcode_e ql_sms_read_msg(uint8_t nSim, uint8_t index, char *buf, uint16_t buf_len,
ql_sms_format_e format)
```

● Parameter

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

index:

[In] Index of SMS message.

buf:

[Out] An array pointer for receiving SMS message.

buf_len:

[In] The array length.

format:

[In] SMS message format. See **Chapter 2.3.4.1** for details.

● Return Value

See **Chapter 2.3.1.1** for details.

2.3.4.1. ql_sms_format_e

The SMS message format is defined as follows:

```
typedef enum
{
    PDU = 0,
    TEXT = 1,
}ql_sms_format_e
```

- **Parameter**

Parameter	Description
<i>PDU</i>	PDU format
<i>TEXT</i>	Text format

2.3.5. ql_sms_read_msg_list

This function reads the list of SMS messages.

- **Prototype**

```
ql_sms_errcode_e ql_sms_read_msg_list(uint8_t nSim, ql_sms_format_e format)
```

- **Parameter**

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

format:

[In] SMS message format. See **Chapter 2.3.4.1** for details.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.6. ql_sms_delete_msg

This function deletes SMS messages.

- **Prototype**

```
ql_sms_errcode_e ql_sms_delete_msg(uint8_t nSim, uint8_t index)
```

- **Parameter**

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

index:

[In] The index of the SMS messages to be deleted.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.7. ql_sms_get_center_address

This function gets the SMS center number.

- **Prototype**

```
ql_sms_errcode_e ql_sms_get_center_address(uint8_t nSim, char* address, uint8_t len)
```

- **Parameter**

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

address:

[Out] An array pointer for receiving the SMS center number.

len:

[In] The array length.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.8. ql_sms_set_center_address

This function sets the SMS center number. If there is no special requirement, it is not recommended to change the SMS center number.

● Prototype

```
ql_sms_errcode_e ql_sms_set_center_address(uint8_t nSim, char* address)
```

● Parameter

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

address:

[In] SMS center number in string format. The prefix "+86" is required.

● Return Value

See **Chapter 2.3.1.1** for details.

2.3.9. ql_sms_set_storage

This function sets the storage location of the SMS message.

● Prototype

```
ql_sms_errcode_e ql_sms_set_storage(uint8_t nSim, ql_sms_stor_e mem1, ql_sms_stor_e mem2, ql_sms_stor_e mem3)
```

● Parameter

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

mem1:

[In] The storage location for SMS messages that have been read and deleted. See **Chapter 2.3.9.1** for details.

mem2:

[In] The storage location for SMS messages to be written and sent. See **Chapter 2.3.9.1** for details.

mem3:

[In] The preferred storage location for SMS messages that have been received. See **Chapter 2.3.9.1** for details.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.9.1. ql_sms_stor_e

The enumeration of SMS message storage location is defined as follows:

```
typedef enum
{
    ME = 1,
    SM = 2,
}ql_sms_stor_e
```

- **Parameter**

Parameter	Description
<i>ME</i>	Mobile equipment (ME)
<i>SM</i>	(U)SIM card

2.3.10. ql_sms_get_storage

This function gets the storage location of SMS message.

- **Prototype**

```
ql_sms_errcode_e ql_sms_get_storage(uint8_t nSim, ql_sms_mem_info_t *mem_info)
```

- **Parameter**

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

mem_info:

[Out] The storage location of SMS message. See **Chapter 2.3.10.1** for details.

- **Return Value**

See **Chapter 2.3.1.1** for details.

2.3.10.1. ql_sms_mem_info_t

The structure of SMS message storage location is defined as follows:

```
typedef struct
{
    ql_sms_stor_e mem1;
    ql_sms_stor_e mem2;
    ql_sms_stor_e mem3;
}ql_sms_mem_info_t
```

- **Parameter**

Type	Parameter	Description
<i>ql_sms_stor_e</i>	<i>mem1</i>	The storage location for SMS messages that have been read and deleted. See Chapter 2.3.9.1 for details.
<i>ql_sms_stor_e</i>	<i>mem2</i>	The storage location for SMS messages to be written and sent. See Chapter 2.3.9.1 for details.
<i>ql_sms_stor_e</i>	<i>mem3</i>	The preferred storage location for SMS messages that have been received. See Chapter 2.3.9.1 for details.

2.3.11. ql_sms_get_storage_info

This function gets the storage status of SMS message in the (U)SIM card and ME.

- **Prototype**

```
ql_sms_errcode_e ql_sms_get_storage_info(uint8_t nSim, ql_sms_stor_info_s *stor_info)
```

- **Parameter**

nSim:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

- 0 (U)SIM card 1
- 1 (U)SIM card 2

stor_info:

[Out] The structure of SMS message storage status. See **Chapter 2.3.11.1** for details.

- Return Value

See **Chapter 2.3.1.1** for details.

2.3.11.1.ql_sms_stor_info_s

The structure of SMS message storage status is defined as follows:

```
typedef struct
{
    uint16_t usedSlotSM;
    uint16_t totalSlotSM;
    uint16_t unReadRecordsSM;
    uint16_t usedSlotME;
    uint16_t totalSlotME;
    uint16_t unReadRecordsME;
    ql_sms_stor_e newSmsStorId;
}ql_sms_stor_info_s
```

- Parameter

Type	Parameter	Description
uint16_t	<i>usedSlotSM</i>	The number of SMS message stored in (U)SIM card (used storage memory)
uint16_t	<i>totalSlotSM</i>	The total number of SMS message that can be stored in (U)SIM card
uint16_t	<i>unReadRecordsSM</i>	The number of unread SMS message in (U)SIM card
uint16_t	<i>usedSlotME</i>	The number of SMS message stored in ME (used storage memory)
uint16_t	<i>totalSlotME</i>	The total number of SMS message that can be stored in ME
uint16_t	<i>unReadRecordsME</i>	The number of unread SMS message in ME
<i>ql_sms_stor_e</i>	<i>newSmsStorId</i>	The storage location of new SMS message. See Chapter 2.3.9.1 for details.

2.3.12. ql_sms_callback_register

This function registers the SMS message callback function.

- **Prototype**

```
void ql_sms_callback_register(ql_sms_event_handler_t cb)
```

- **Parameter**

cb:

[In] Callback function. See **Chapter 2.3.12.1** for details.

- **Return Value**

None

2.3.12.1. ql_sms_event_handler_t

This function defines the callback function for SMS message events.

- **Prototype**

```
typedef void (*ql_sms_event_handler_t)(uint8_t sim_id, int event_id, void *ctx)
```

- **Parameter**

sim_id:

[In] The (U)SIM card used. If the module only supports one (U)SIM interface, this parameter can only be set to 0.

0 (U)SIM card 1

1 (U)SIM card 2

event_id:

[In] SMS message events. See **Chapter 2.3.12.2** for details.

ctx:

[In] The pointer of inputting parameters for the callback function.

- **Return Value**

None

2.3.12.2.ql_sms_event_id_e

The enumeration of SMS message events is defined as follows:

```
typedef enum
{
    QL_SMS_INIT_OK_IND = 1 | (QL_COMPONENT_SMS << 16),
    QL_SMS_NEW_MSG_IND,
    QL_SMS_LIST_IND,
    QL_SMS_LIST_END_IND,
    QL_SMS_MEM_FULL_IND,
}ql_sms_event_id_e
```

● Parameter

Parameter	Description
<i>QL_SMS_INIT_OK_IND</i>	Successfully initialization of SMS
<i>QL_SMS_NEW_MSG_IND</i>	New SMS message
<i>QL_SMS_LIST_IND</i>	List of SMS message
<i>QL_SMS_LIST_END_IND</i>	The list of SMS message has been read
<i>QL_SMS_MEM_FULL_IND</i>	The storage memory of SMS message is full

3 SMS Demo

This chapter mainly introduces how to use the above APIs to develop SMS functions on the Application side.

3.1. Demo Introduction

sms_demo.c, provided in the SDK of Quectel EC200U and EG91xU family modules, is located in the directory of *components\ql-application\sms*. The demo contains some functions including sending, reading and deleting SMS messages. The following example shows how to use the API to send English and Chinese SMS messages.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#include "ql_api_common.h"
#include "ql_api_osi.h"
#include "ql_api_sms.h"
#include "ql_log.h"

ql_task_t sms_task = NULL;
ql_sem_t sms_init_sem = NULL;

void user_sms_event_callback(uint8_t nSim, int event_id, void *ctx)
{
    switch(event_id)
    {
        case QL_SMS_INIT_OK_IND:
        {
            QL_SMS_LOG("QL_SMS_INIT_OK_IND");
            ql_rtos_semaphore_release(sms_init_sem);
            break;
        }
        case QL_SMS_NEW_MSG_IND:
        {
            ql_sms_new_s *msg = (ql_sms_new_s *)ctx;
```

```

        QL_SMS_LOG("sim=%d, index=%d, storage memory=%d", nSim, msg->index,
msg->mem);
        break;
    }
    case QL_SMS_MEM_FULL_IND:
    {
        ql_sms_new_s *msg = (ql_sms_new_s *)ctx;
        QL_SMS_LOG("QL_SMS_MEM_FULL_IND sim=%d, memory=%d",nSim,msg->mem);
        break;
    }
    default :
        break;
}
}

void sms_demo_task(void * param)
{
    uint8_t nSim = 0;
    QL_SMS_LOG("enter");
    ql_sms_callback_register(user_sms_event_callback);

    //wait sms ok
    if(ql_rtos_semaphore_wait(sms_init_sem, QL_WAIT_FOREVER)){
        QL_SMS_LOG("Waiting for SMS init timeout");
    }

    //Send English text message
    if(QL_SMS_SUCCESS == ql_sms_send_msg(nSim,"+86189xxxxxxx","Hello,marvin", GSM)){
        QL_SMS_LOG("send sms OK");
    }else{
        QL_SMS_LOG("send sms FAIL");
    }
    //Send messages in Chinese and English. (Need use UTF8 encoding to open sms_demo.c for
chinese.)
    if(QL_SMS_SUCCESS == ql_sms_send_msg(nSim,"+86189xxxxxxx","hello,你好", UCS2)){
        QL_SMS_LOG("send pdu sms OK");
    }else{
        QL_SMS_LOG("send pdu sms FAIL");
    }
    ql_rtos_task_delete(NULL);
}

```

```

QIOSStatus ql_sms_app_init(void)
{
    QIOSStatus err = QL_OSI_SUCCESS;

    err = ql_rtos_task_create(&sms_task, 4096, APP_PRIORITY_NORMAL, "SMS_TASK",
    sms_demo_task, NULL, 2);
    if(err != QL_OSI_SUCCESS)
    {
        QL_SMS_LOG("sms_task created failed, ret = 0x%x", err);
    }

    err = ql_rtos_semaphore_create(&sms_init_sem, 0);
    if(err != QL_OSI_SUCCESS)
    {
        QL_SMS_LOG("sms_init_sem created failed, ret = 0x%x", err);
    }

    return err;
}
    
```

3.2. Demo Autorun

The Demo is not enabled by default in *ql_init_demo_thread()*. As shown in the figure below, uncomment the codes when you run a test. In addition, you need to change the target phone number of the *ql_sms_send_msg()* in the demo to the your own phone number.

```

:
:
: #ifdef QL_APP_FEATURE_SMS
:     //ql_sms_app_init();
: #endif
: #ifdef QL_APP_FEATURE_VOICE_CALL
:     //ql_voice_call_app_init();
: #endif
: #ifdef QL_APP_FEATURE_VOLTE
:     //ql_volte_app_init();
: #endif
:
:
    
```

4 Appendix References

Table 3: Related Document

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen_CSDK_Quick_Start_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
App	Application
GSM	Global System for Mobile Communications
IoT	Internet of Things
LTE	Long-Term Evolution
ME	Mobile Equipment
PDU	Packet Data Unit
RTOS	Real-Time Operating System
SDK	Software Development Kit
(U)SIM	(Universal) Subscriber Identity Module
SMS	Short Message Service
UCS	Universal Character Set
UTF	Unicode Transformation Format